Chapter 11 Warm-Up: A 2D Particle Class

Prof. Jordan C. Hanson

1 A Moving Particle

A particle in two dimensions has position (x, y) and velocity (v_x, v_y). For constant velocity over a time step Δt,

$$x_{\text{new}} = x + v_x \Delta t, \qquad y_{\text{new}} = y + v_y \Delta t.$$

For

$$(x, y) = (1, 2), \qquad (v_x, v_y) = (3, -1),$$

find the position after $\Delta t = 0.5$:

$$x_{\text{new}} = _____, \qquad y_{\text{new}} = _____.$$

2 Build the Class

Create a class that stores the particle's state:

```
class Particle2D:
    def __init__(self,x,y,vx,vy):
        self.x = x
        self.y = y
        self.vx = vx
        self.vy = vy
```

Create one particle:

```
p1 = Particle2D(1, 2, 3, -1)
```

Identify the four instance attributes of `p1`.

3 Change the State

Add this method inside the class:

```
def update(self, dt):
    self.x += self.vx * dt
    self.y += self.vy * dt
```

Then run

```
p1.update(0.5)
print(p1.x, p1.y)
```

Does the output agree with your hand calculation? Why do `vx` and `vy` remain unchanged?

4 Advance Through Time

Reset the particle:

```
p1 = Particle2D(1, 2, 3, -1)

dt = 0.1
for i in range(10):
    p1.update(dt)
```

How much total time has passed?

$$t = _____$$

Predict the final position:

$$x = _____, \qquad y = _____.$$

Print `p1.x` and `p1.y` to check.

5 Independent Objects

Create another particle:

```
p2 = Particle2D(-2, 0, 1, 2)
```

Run

```
p1.update(0.5)
p2.update(0.5)
```

Why does updating `p1` not change `p2`?

Each object has its own **state**: its own values of `x`, `y`, `vx`, and `vy`.

6 Challenge: A New Method

Add a method that returns

$$v = \sqrt{v_x^2 + v_y^2}.$$

Begin the file with

```
import math
```

and complete

```
def speed(self):
    return ...
```

Test it using

```
print(p1.speed())
```

Why should `speed()` return a value while `update()` changes the object's state?